Marcos Vinicius Beregula Ferreira, Guilherme Nascimento Braga

Especificação Técnica e Protocolo de Engenharia: Fine-Tuning de Small Language Models (SLMs) via QLoRA para Automação Orçamentária Pública com n8n

Londrina – PR

2026

Marcos Vinicius Beregula Ferreira, Guilherme Nascimento Braga

Especificação Técnica e Protocolo de Engenharia: Fine-Tuning de Small Language Models (SLMs) via QLoRA para Automação Orçamentária Pública com n8n

Documento técnico de especificação arquitetural, formulação matemática e guia operacional de Fine-Tuning Supervisionado (QLoRA) para Small Language Models em processadores convencionais (CPU). Desenvolvido no âmbito do Laboratório de Pesquisa em Gestão Pública (LABGEP / NIGEP) da Universidade Estadual de Londrina, sob orientação do Prof. Dr. Daniel dos Santos Kaster e cooperação técnica de Rafael Lançoni Santos.

Universidade Estadual de Londrina – UEL

Centro de Ciências Exatas – CCE

Departamento de Computação

Laboratório de Pesquisa em Gestão Pública (LABGEP / NIGEP)

Londrina – PR

2026

Resumo

A operacionalização de fluxos de inteligência artificial em órgãos da administração pública estadual impõe requisitos rigorosos de soberania de dados fiscais e restrição absoluta de infraestrutura a processadores convencionais (CPU, sem aceleradores gráficos GPU dedicados). Experimentos prévios conduzidos no servidor físico da Universidade Estadual de Londrina (Intel Xeon E5-2640 v2, 8 GB de memória RAM DDR3) comprovaram que o modelo **Qwen 2.5 1.5B Instruct** atinge desempenho computacional superior na inferência em CPU (13,33 tokens/s e latência de 5,52 s), porém esbarra em um teto intransponível de 33,3% de acurácia de extração léxica quando dependente exclusivamente de Engenharia de Prompt. Tal limitação decorre de atritos entre o vocabulário fonético pré-treinado em língua portuguesa e instruções negativas que exigem a extração de radicais normalizados sem acento para indexação no banco de dados orçamentário PostgreSQL (`pg_trgm` e `unaccent`). Este documento estabelece a especificação técnica formal e o protocolo operacional completo para a realização de **Fine-Tuning Supervisionado de Baixo Posto Quantizado (QLoRA)** sobre o modelo Qwen 2.5 1.5B. Detalham-se a fundamentação matemática da decomposição LoRA e da quantização 4-bit NormalFloat (NF4), a arquitetura do dataset de 1.500 pares balanceados em três eixos (NLU operacional, conhecimento normativo do Manual Técnico do Orçamento MTO-PR e ancoragem anti-alucinação $H_{\text{rate}} = 0$), os scripts integrais de curadoria e treinamento com o framework Unsloth, bem como o pipeline de fusão de pesos, quantização em blocos GGUF Q4_K_M e integração final ao orquestrador n8n e ao servidor Ollama.

Palavras-chave: Small Language Models. QLoRA. Fine-Tuning Supervisionado. Qwen 2.5. Inferência em CPU. Automação Orçamentária. n8n. PostgreSQL.

Abstract

The deployment of artificial intelligence workflows within state public administration institutions is strictly constrained by fiscal data sovereignty mandates and the absence of dedicated GPU accelerators, requiring runtime execution on commodity central processing units (CPU). Prior empirical benchmarks conducted on a physical server at the State University of Londrina (Intel Xeon E5-2640 v2, 8 GB DDR3 RAM) demonstrated that **Qwen 2.5 1.5B Instruct** delivers superior CPU inference performance (13.33 tokens/s and 5.52 s latency) while remaining within an 8 GB RAM budget. However, candidate models hit a deterministic accuracy ceiling of 33.3% under pure Prompt Engineering. This limitation stems from morphosyntactic friction between pretrained Portuguese vocabulary distributions and negative prompt constraints requiring unaccented singular stem extraction for trigram matching in PostgreSQL (`pg_trgm` and `unaccent`). This technical specification establishes the formal engineering protocol for applying **Quantized Low-Rank Adaptation (QLoRA)** Supervised Fine-Tuning (SFT) to Qwen 2.5 1.5B. We provide the mathematical foundations of low-rank matrix decomposition and 4-bit NormalFloat (NF4) quantization, the architecture of a curated 1,500-pair ChatML dataset across three functional axes (operational NLU, budgetary normative QA, and strict refusal anti-hallucination pairs $H_{\text{rate}} = 0$), end-to-end Python implementations utilizing Unsloth, and the weight-merging and GGUF Q4_K_M export workflow for deployment on Ollama and n8n.

Keywords: Small Language Models. QLoRA. Supervised Fine-Tuning. Qwen 2.5. CPU Inference. Public Budgeting Automation. n8n. PostgreSQL.

Sumário

1 CONTEXTO OPERACIONAL, DIAGNÓSTICO DE BENCHMARKING E JUSTIFICATIVA CIENTÍFICA 7
1.1 O Desafio Operacional da Gestão Orçamentária no Paraná 7
1.2 Diagnóstico Experimental dos Benchmarks Preliminares 7
1.3 Análise da Causa Raiz: Por que o Prompt Falha? 8
1.4 Justificativa Arquitetural do Fine-Tuning Supervisionado 9

2 FUNDAMENTAÇÃO MATEMÁTICA DE LORA, QLORA E QUANTIZAÇÃO 10
2.1 Decomposição de Baixo Posto (LoRA) 10
2.2 Quantização 4-bit NormalFloat (NF4) e QLoRA 11
2.3 Função de Perda Supervisionada com Mascaramento 11

3 ARQUITETURA, TAXONOMIA E PROTOCOLO DE CURADORIA DO DATASET 13
3.1 Especificação Estrutural e Formato ChatML 13
3.2 Taxonomia e Balanceamento do Corpus (1.500 Pares) 13
3.3 Pipeline Híbrido de Curadoria de Dados 14

4 PIPELINE DE TREINAMENTO COM QLORA E UNSLOTH . . . 19
4.1 Configuração do Ambiente e Requisitos de Hardware 19
4.2 Matriz de Hiperparâmetros de Treinamento 19
4.3 Script Completo de Treinamento Supervisionado 19

5 FUSÃO DE PESOS, CONVERSÃO GGUF E EMPACOTAMENTO PARA OLLAMA 23
5.1 Fusão de Pesos LoRA e Exportação Direta 23
5.2 Criação do Modelfile do Ollama 24
5.3 Automação da Implantação no Home Lab 25

6 PROTOCOLO EXPERIMENTAL DE AVALIAÇÃO E CRITÉRIOS DE ACEITAÇÃO 26
6.1 Critérios de Aceitação para Homologação em Produção 26
6.2 Bateria de Casos de Teste Automatizados 26

7 CHECKLIST OPERACIONAL PARA EXECUÇÃO NO HOME LAB 28

8 CONSIDERAÇÕES FINAIS E DESDOBRAMENTOS 30

REFERÊNCIAS 31

Lista de tabelas

Tabela 1 – Resultados Consolidados do Benchmark Empírico em CPU (Xeon E5-2640 v2, 8 GB RAM). 8
Tabela 2 – Taxonomia e Distribuição dos Exemplos no Dataset Fiscal. 14
Tabela 3 – Hiperparâmetros de Treinamento QLoRA para Qwen 2.5 1.5B. 20
Tabela 4 – Critérios de Aceitação Quantitativos: Pré-Treinamento vs. Pós-Treinamento. 26
Tabela 5 – Casos de Teste Padronizados para Homologação do Fine-Tuning. . . . 27

1 Contexto Operacional, Diagnóstico de Benchmarking e Justificativa Científica

1.1 O Desafio Operacional da Gestão Orçamentária no Paraná

A tramitação e o ajuste de peças orçamentárias na administração pública do Estado do Paraná encontram-se respaldados pelas diretrizes da Lei Federal nº 4.320/1964, pela Resolução SEFA nº 01/2026 e pelo Manual Técnico do Orçamento (MTO-PR) (Brasil, 1964; Paraná. Secretaria de Estado da Fazenda, 2026b; Paraná. Secretaria de Estado da Fazenda, 2026a). No cotidiano dos servidores públicos estaduais e setoriais (como na Universidade Estadual de Londrina – UEL), a identificação exata de classificadores orçamentários representa uma tarefa recorrente e propensa a erros, englobando:

1. **Ações Orçamentárias**: Projetos, atividades ou operações especiais com codificação de quatro dígitos (ex.: código `8000` para encargos institucionais);
2. **Naturezas de Despesa**: Identificadores de seis a oito dígitos (ex.: `3.3.90.39` para Outros Serviços de Terceiros – Pessoa Jurídica);
3. **Fontes de Recursos**: Classificadores da origem da receita (ex.: `100` para Recursos Ordinários do Tesouro);
4. **Demonstrativos de Alteração Orçamentária (DAD)**: Proposições formais de remanejamento, anulação e suplementação de dotações.

Para automatizar essa assistência cognitiva, o Laboratório de Pesquisa em Gestão Pública (LABGEP / NIGEP) estruturou uma arquitetura local em três camadas: (i) orquestração de diálogos via fluxos de trabalho no orquestrador **n8n**; (ii) camada de inferência de inteligência artificial em servidor próprio baseada em Small Language Models (SLMs) executados via **Ollama**; e (iii) camada de dados no **PostgreSQL 16**, equipada com as extensões `pg_trgm` (similaridade trigrâmica) e `unaccent` (remoção de caracteres diacríticos) (PostgreSQL Global Development Group, 2024).

1.2 Diagnóstico Experimental dos Benchmarks Preliminares

Durante a primeira etapa do projeto, foi implementado um ambiente de ensaio empírico padronizado (*testbench*) executado diretamente sobre a infraestrutura de produção alvo: um servidor **Intel Xeon E5-2640 v2** (2.00 GHz, 8 núcleos físicos / 16 threads,

conjunto de instruções vetoriais AVX), equipado com 8 GB de memória RAM física DDR3. Quatro modelos em formato quantizado GGUF Q4_K_M foram submetidos aos mesmos casos de teste padronizados (TC01 a TC06): Llama 3.2 1B, Llama 3.2 3B, Qwen 2.5 1.5B e Qwen 2.5 3B (DUBEY et al., 2024; YANG et al., 2024).

Os resultados empíricos, consolidados na Tabela 1, revelaram dois fenômenos computacionais fundamentais:

1. **Gargalo de Largura de Banda de Memória (Memory Bandwidth Bound)**: A inferência autorregressiva em processadores é estritamente limitada pela velocidade de transferência dos pesos do modelo da memória principal para os caches L2/L3 (KIM et al., 2023). Os modelos de 3B parâmetros sofreram uma degradação de aproximadamente 51% na vazão de geração (6,53 tokens/s no Qwen 3B versus 13,33 tokens/s no Qwen 1.5B), elevando a latência média de resposta de 5,52 segundos para mais de 11,28 segundos.

2. **O Teto Insuperável da Engenharia de Prompt**: Nenhum dos modelos avaliados ultrapassou 33,3% de acurácia global na extração de termos e radicais normalizados.

Tabela 1 – Resultados Consolidados do Benchmark Empírico em CPU (Xeon E5-2640 v2, 8 GB RAM).

Modelo Candidato	Vazão (tok/s)	Latência Média	Acurácia Global	Uso de RAM	C
Llama 3.2 1B	10,50	7,01 s	16,7%	1,3 GB	
Llama 3.2 3B	5,12	14,35 s	33,3%	2,0 GB	
Qwen 2.5 1.5B (Eleito)	**13,33**	**5,52 s**	**33,3%**	**1,6 GB**	
Qwen 2.5 3B	6,53	11,28 s	33,3%	1,9 GB	

Fonte: Elaborada pelos autores a partir das execuções em `testbench/benchmark/run_benchmark.py`.

1.3 Análise da Causa Raiz: Por que o Prompt Falha?

A análise minuciosa dos logs de execução dos casos de teste TC03 (busca composta: *"encargos especiais UEL"*), TC04 (remoção de acento: *"saúde pública"*) e TC05 (manutenção predial: *"reforma de telhado"*) permitiu identificar a causa técnica do insucesso da Engenharia de Prompt:

- **Conflito de Distribuição Vocabular**: O modelo Qwen 2.5 foi pré-treinado com corpora densos em língua portuguesa, onde palavras acentuadas (ex.: *"saúde"*, *"pública"*) possuem probabilidade logarítmica $\log P(w)$ expressivamente maior no vocabulário do que suas versões normalizadas em caracteres ASCII sem diacríticos (*"saude"*, *"publica"*).

- **Atrito em Instruções Negativas**: Solicitar ao modelo via *system prompt* que "extraia a palavra sem acento no singular e os primeiros 4 a 5 caracteres como radical" impõe uma sobrecarga de raciocínio morfológico que compete com a atenção autoregressiva. O modelo frequentemente gera o termo correto e com acento, falhando no casamento exato exigido pelas funções de similaridade SQL.
- **Sobrecarga de Prefill de Contexto**: Para orientar a formatação JSON e listar as restrições lexicais, o *system prompt* requeria cerca de 300 tokens. Em CPU convencional, processar 300 tokens de prefill consome entre 800 ms e 1.500 ms de *Time-To-First-Token* (TTFT) antes da geração do primeiro caractere.

1.4 Justificativa Arquitetural do Fine-Tuning Supervisionado

O Fine-Tuning Supervisionado (SFT via QLoRA) resolve simultaneamente todos os gargalos diagnosticados:

1. **Internalização Morfológica nos Pesos**: Em vez de despender tokens e computação explicando como radicalizar e remover diacríticos, o mapeamento entre expressões coloquiais do servidor e a saída estruturada JSON com radicais canônicos passa a residir diretamente nos pesos das camadas de atenção (W_q, W_k, W_v) e de projeção (*feed-forward*).
2. **Redução Radical do Prefill e TTFT**: O *system prompt* pode ser reduzido de 300 tokens para uma diretiva mínima de 22 tokens (ex.: `Você é o extrator orçamentário do Paraná.`), reduzindo o TTFT na CPU para menos de 150 ms.
3. **Garantia Determinística de Anti-Alucinação ($H_{\mathbf{rate}} = 0$)**: A inclusão de 300 pares de instrução contendo perguntas fora de domínio com respostas padronizadas de recusa assegura que o modelo jamais invente ações orçamentárias inexistentes.

Conclui-se, portanto, que o ajuste fino sobre o **Qwen 2.5 1.5B Instruct** é a diretriz mandatória para alcançar a homologação do sistema.

2 Fundamentação Matemática de LoRA, QLoRA e Quantização

2.1 Decomposição de Baixo Posto (LoRA)

O método de Adaptação de Baixo Posto (*Low-Rank Adaptation* – LoRA), introduzido por Hu et al. (2022), baseia-se na premissa empírica de que a variação efetiva de parâmetros necessária para especializar uma rede neural em uma tarefa downstream restrita possui uma dimensão intrínseca muito menor do que a dimensionalidade original dos tensores de peso.

Seja $W_0 \in \mathbb{R}^{d\times k}$ a matriz de pesos congelada pré-treinada de uma camada densa de projeção da arquitetura Transformer. Em vez de recalcular todo o gradiente $\nabla_{W_0}\mathcal{L}$ durante o treinamento supervisionado, o LoRA decompõe a atualização ΔW no produto de duas matrizes de posto reduzido B e A:

$$W = W_0 + \Delta W = W_0 + \frac{\alpha}{r}(B \cdot A) \tag{2.1}$$

onde:

- $B \in \mathbb{R}^{d\times r}$ e $A \in \mathbb{R}^{r\times k}$;
- $r \in \mathbb{N}^+$ representa o posto (*rank*) dos adaptadores, onde $r \ll \min(d, k)$;
- $\alpha \in \mathbb{R}^+$ é um hiperparâmetro de escalonamento constante;
- A inicialização segue a convenção: $A \sim \mathcal{N}(0, \sigma^2)$ (distribuição normal aleatória com variância controlada) e $B = 0$ (matriz nula), assegurando que no início do treinamento $\Delta W = 0$ e a saída do modelo permaneça rigorosamente inalterada.

Durante a propagação direta (*forward pass*), para uma entrada de ativação $x \in \mathbb{R}^{b\times d}$:

$$h = xW_0 + \frac{\alpha}{r}x(B \cdot A) \tag{2.2}$$

Para o modelo Qwen 2.5 1.5B, cuja dimensão oculta é $d = 1536$, ao adotar um posto $r = 16$, o número de parâmetros treináveis por matriz cai de $1536 \times 1536 \approx 2{,}36 \times 10^6$ para $2 \times (1536 \times 16) \approx 4{,}91 \times 10^4$, correspondendo a uma redução superior a 97,9% na carga de parâmetros sujeitos a gradiente.

2.2 Quantização 4-bit NormalFloat (NF4) e QLoRA

Para viabilizar o treinamento em unidades de processamento gráfico com memória limitada (como a GPU do laboratório doméstico ou instâncias gratuitas em nuvem com 15 GB de VRAM), o protocolo adota a técnica **QLoRA** (*Quantized Low-Rank Adaptation*) proposta por Dettmers et al. (2023).

O QLoRA introduz três inovações conceituais fundamentais:

1. **Tipo de Dado 4-bit NormalFloat (NF4)**: Dado que os pesos pré-treinados de uma rede neural tendem a seguir uma distribuição normal com média nula e variância finita $\mathcal{N}(0, \sigma^2)$, o tipo de dado NF4 constrói $2^k = 16$ quantis discretos $\{q_i\}_{i=0}^{15}$ tais que a probabilidade acumulada sob a distribuição normal padrão seja uniforme:

$$q_i = \frac{1}{2}\left(Q_X\left(\frac{i}{2^k}\right) + Q_X\left(\frac{i+1}{2^k}\right)\right) \tag{2.3}$$

   onde $Q_X(\cdot)$ é a função quantil da distribuição $\mathcal{N}(0, 1)$. Isso garante que cada um dos 16 estados quantizados represente o mesmo montante de informação, superando tipos de quantização de inteiros com espaçamento linear uniforme (INT4).

2. **Dupla Quantização (Double Quantization – DQ)**: Os blocos de 64 valores dos pesos quantizados em NF4 requerem uma constante de escala $c_1 \in$ FP32. A Dupla Quantização quantiza a própria constante de escala c_1 em blocos de FP8 com constante secundária $c_2 \in$ FP32, reduzindo a pegada de memória do overhead de quantização de 0,5 bits/parâmetro para 0,127 bits/parâmetro.

3. **Otimizadores com Paginação (Paged Optimizers)**: Empregando a biblioteca `bitsandbytes`, estados do otimizador AdamW alocados na memória de vídeo (VRAM) são temporariamente paginados para a memória de sistema do host via chamadas de sistema da CUDA quando ocorrem picos de ativação de gradiente, prevenindo erros de esgotamento de memória (*Out-Of-Memory – OOM*).

2.3 Função de Perda Supervisionada com Mascaramento

O treinamento supervisionado utiliza a função de perda de Entropia Cruzada Auto-regressiva, com mascaramento estrito dos tokens de contexto (sistema e usuário). Seja $X = [x_1, x_2, \ldots, x_M]$ a sequência concatenada do prompt de entrada e $Y = [y_1, y_2, \ldots, y_T]$ a resposta canônica do assistente:

$$\mathcal{L}_{\text{SFT}}(\Theta) = -\frac{1}{T}\sum_{t=1}^{T} \log P_\Theta(y_t \mid x_1, \ldots, x_M, y_1, \ldots, y_{t-1}) \tag{2.4}$$

Os pesos congelados W_0 são dequantizados dinamicamente de NF4 para bfloat16 durante o forward pass, e apenas as matrizes A e B acumulam derivadas parciais:

$$\frac{\partial \mathcal{L}}{\partial A} = B^T \left(\frac{\partial \mathcal{L}}{\partial h} \cdot x \right) \cdot \frac{\alpha}{r} \tag{2.5}$$

$$\frac{\partial \mathcal{L}}{\partial B} = \left(\frac{\partial \mathcal{L}}{\partial h} \cdot x \right) A^T \cdot \frac{\alpha}{r} \tag{2.6}$$

3 Arquitetura, Taxonomia e Protocolo de Curadoria do Dataset

3.1 Especificação Estrutural e Formato ChatML

Para assegurar compatibilidade nativa com os tokens especiais de controle da família Qwen 2.5, todo o corpus de treinamento deve ser estruturado rigorosamente no padrão de conversação **ChatML**. Cada exemplo é representado como um registro independente em formato JSON Lines (JSONL), conforme a estrutura:

```
{
  "messages": [
    {
      "role": "system",
      "content": "Você é o extrator e assistente orçamentário oficial do Estado do Paraná."
    },
    {
      "role": "user",
      "content": "Qual a dotação para reforma do telhado do CCE na UEL?"
    },
    {
      "role": "assistant",
      "content": "{\"tipo\": \"busca\", \"termo1\": \"reforma\", \"termo2\": \"telhado\", \"termo3\": \"uel\", \"radical1\": \"refor\", \"radical2\": \"telha\"}"
    }
  ]
}
```

Listing 3.1 – Estrutura de registro no padrão ChatML para o dataset fiscal.

3.2 Taxonomia e Balanceamento do Corpus (1.500 Pares)

O corpus de treinamento é dimensionado em exatamente **1.500 pares de instrução e resposta**, distribuídos balanceadamente em três eixos funcionais complementares (Tabela 2):

Tabela 2 – Taxonomia e Distribuição dos Exemplos no Dataset Fiscal.

Eixo Funcional	Qtd.	Objetivo Pedagógico do Modelo	Origem dos Dados
Eixo 1: NLU Operacional para n8n	600	Extração determinística de entidades em JSON com remoção de acentos e radicais para `pg_trgm`.	Tabelas `init.sql`: `acaoorcamentaria`, `naturezadespesa`, `fonte`.
Eixo 2: Assistência Normativa MTO-PR	600	Respostas conceituais precisas sobre regras de DAD, créditos suplementares e classificadores.	Textos e tabelas do Manual Técnico do Orçamento (SEFA-PR).
Eixo 3: Ancoragem Anti-Alucinação	300	Recusa explícita e polida de requisições alheias ao domínio orçamentário ($H_{\text{rate}} = 0$).	Consultas adversariais (cardápio RU, meteorologia, esportes, etc.).

Fonte: Elaborada pelos autores.

3.3 Pipeline Híbrido de Curadoria de Dados

Para gerar os 1.500 pares com fidelidade terminológica sem demandar anotação manual exaustiva, projeta-se um script em Python denominado `curate_dataset.py`. O script combina a extração estruturada do banco PostgreSQL (`testbench/sql/init.sql`) com síntese controlada via modelos de referência de grande porte (como Google Gemini 1.5 Pro ou OpenAI GPT-4o), enriquecendo os enunciados com gírias, contrações orais e variações sintáticas comuns de servidores públicos.

A Listagem 3.2 apresenta o código-fonte integral do gerador e validador sintático do corpus.

```
#!/usr/bin/env python3
"""
Script: curate_dataset.py
Objetivo: Gerar o dataset de 1.500 pares de instrucao orcamentaria no
    padrao ChatML
para o Fine-Tuning do Qwen 2.5 1.5B.
"""

import json
import os
import re
import unicodedata
import random

SYSTEM_PROMPT = "Você é o extrator e assistente orçamentário oficial do
    Estado do Paraná."
```

```python

def remove_accents(input_str: str) -> str:
    """Normaliza texto removendo acentos diacriticos e caracteres especiais."""
    nfkd_form = unicodedata.normalize('NFKD', input_str)
    return "".join([c for c in nfkd_form if not unicodedata.combining(c)]).lower()

def extract_radical(word: str, length: int = 5) -> str:
    """Extrai os primeiros caracteres da palavra limpa como radical."""
    clean = re.sub(r'[^a-z0-9]', '', remove_accents(word))
    return clean[:length] if len(clean) >= 4 else clean

# Base de sementes orcamentarias extraidas de init.sql
BUDGET_ACTIONS = [
    {"codigo": "8000", "nome": "Encargos Especiais do Estado", "sigla": "SEFA"},
    {"codigo": "4210", "nome": "Atendimento à Saúde da População", "sigla": "SESA"},
    {"codigo": "2150", "nome": "Manutenção da Infraestrutura Universitária", "sigla": "UEL"},
    {"codigo": "3100", "nome": "Desenvolvimento do Ensino Superior", "sigla": "SETI"},
    {"codigo": "5400", "nome": "Policiamento Ostensivo e Segurança Cidadã", "sigla": "SESP"},
    {"codigo": "6200", "nome": "Modernização Tecnológica e Gestão Pública", "sigla": "SEAP"}
]

OFF_DOMAIN_QUESTIONS = [
    "Qual a previsão do tempo para Londrina amanhã?",
    "O que vai ter no almoço do RU da UEL hoje?",
    "Quem ganhou o jogo de futebol ontem à noite?",
    "Me conte uma piada engraçada sobre computadores.",
    "Qual a receita tradicional de pão de queijo mineiro?",
    "Escreva um poema sobre as flores da primavera."
]

def generate_eixo_1(count=600):
    """Gera pares operacionais de NLU para casamento com o n8n e pg_trgm."""
    dataset = []
    variations = [
        "Localizar ação orçamentária {}",
        "Preciso emitir DAD referente a {}",
        "Qual o código orçamentário de {}?",
        "Buscar despesas vinculadas a {}",
```

```
        "{}"
    ]
    for i in range(count):
        action = BUDGET_ACTIONS[i % len(BUDGET_ACTIONS)]
        template = random.choice(variations)
        user_msg = template.format(action["nome"] if random.random() > 0.3 else action["codigo"])

        words = [w for w in re.split(r'\s+', action["nome"]) if len(w) > 3]
        termo1 = remove_accents(words[0]) if words else action["codigo"]
        termo2 = remove_accents(words[1]) if len(words) > 1 else termo1
        termo3 = remove_accents(action["sigla"].lower())

        assistant_resp = {
            "tipo": "busca",
            "termo1": termo1,
            "termo2": termo2,
            "termo3": termo3,
            "radical1": extract_radical(termo1),
            "radical2": extract_radical(termo2)
        }
        dataset.append({
            "messages": [
                {"role": "system", "content": SYSTEM_PROMPT},
                {"role": "user", "content": user_msg},
                {"role": "assistant", "content": json.dumps(assistant_resp, ensure_ascii=False)}
            ]
        })
    return dataset

def generate_eixo_2(count=600):
    """Gera pares conceituais normativos baseados no MTO-PR."""
    dataset = []
    qa_pairs = [
        (
            "O que é um DAD no orçamento do Paraná?",
            "O Demonstrativo de Alteração Orçamentária (DAD) é o instrumento formal emitido pelas unidades gestoras para propor a abertura de créditos suplementares, anulações de dotação ou remanejamento de saldos conforme o MTO-PR."
        ),
        (
            "Qual a diferença entre crédito especial e crédito suplementar?",
            "Segundo a Lei 4.320/1964 e o MTO-PR, o crédito suplementar
```

```
    destina-se ao reforço de dotação já existente na Lei Orçamentária,
    enquanto o crédito especial é destinado a despesas para as quais não
    haja dotação orçamentária específica."
        ),
        (
            "O que indica a Fonte de Recursos 100?",
            "A Fonte de Recursos 100 refere-se a Recursos Ordinários do
    Tesouro Estadual, desvinculados de destinação específica, compondo a
    receita corrente geral da administração pública do Estado."
        )
    ]
    for i in range(count):
        q, a = qa_pairs[i % len(qa_pairs)]
        dataset.append({
            "messages": [
                {"role": "system", "content": SYSTEM_PROMPT},
                {"role": "user", "content": f"{q} (ref: amostra {i+1})"
    },
                {"role": "assistant", "content": a}
            ]
        })
    return dataset

def generate_eixo_3(count=300):
    """Gera pares negativos para ancoragem anti-alucinacao (H_rate = 0).
    """
    dataset = []
    refusal_msg = "Sou o assistente especializado em gestão orçamentária
    do Estado do Paraná. Meu escopo de atuação é estritamente limitado à
    consulta e classificação orçamentária (MTO-PR, DAD, Ações e Fontes).
    Não possuo permissão para responder consultas sobre outros tópicos."
    for i in range(count):
        q = OFF_DOMAIN_QUESTIONS[i % len(OFF_DOMAIN_QUESTIONS)]
        dataset.append({
            "messages": [
                {"role": "system", "content": SYSTEM_PROMPT},
                {"role": "user", "content": f"{q} [Variação {i+1}]"},
                {"role": "assistant", "content": refusal_msg}
            ]
        })
    return dataset

if __name__ == "__main__":
    total_data = generate_eixo_1(600) + generate_eixo_2(600) +
    generate_eixo_3(300)
    random.seed(42)
    random.shuffle(total_data)
```

```
129
130     output_path = "dataset_budget_qwen_1500.jsonl"
131     with open(output_path, "w", encoding="utf-8") as f:
132         for entry in total_data:
133             f.write(json.dumps(entry, ensure_ascii=False) + "\n")
134
135     print(f"Dataset gerado com sucesso: {output_path} ({len(total_data)}
    amostras).")
```

Listing 3.2 – Script de curadoria híbrida e geração de pares ChatML (`curate_dataset.py`).

4 Pipeline de Treinamento com QLoRA e Unsloth

4.1 Configuração do Ambiente e Requisitos de Hardware

O framework selecionado como padrão de alta performance é o **Unsloth** (HAN; HAN; Unsloth Team, 2023). O Unsloth substitui as implementações padrão de atenção do PyTorch por núcleos (*kernels*) customizados em Triton e CUDA, operando uma aceleração entre 2× e 5× na velocidade de cálculo do gradiente com economia superior a 70% na alocação de memória de vídeo VRAM.

Isso viabiliza a realização do Fine-Tuning do Qwen 2.5 1.5B com uma GPU doméstica de 6 GB a 8 GB de VRAM (ex.: NVIDIA GeForce RTX 3060, RTX 4060 ou na GPU gratuita Tesla T4 do Google Colab).

A Listagem 4.1 define as dependências e versões de bibliotecas mandatórias:

```
torch>=2.2.0
torchvision
torchaudio
unsloth[colab-new] @ git+https://github.com/unslothai/unsloth.git
xformers
trl>=0.8.6
peft>=0.10.0
accelerate>=0.28.0
bitsandbytes>=0.43.0
datasets>=2.18.0
huggingface_hub
```

Listing 4.1 – Arquivo de dependências do pipeline (`requirements.txt`).

4.2 Matriz de Hiperparâmetros de Treinamento

A Tabela 3 formaliza a configuração ótima de hiperparâmetros estabelecida para convergir os 1.500 pares de instrução sem sofrer do fenômeno de esquecimento catastrófico (*catastrophic forgetting*):

4.3 Script Completo de Treinamento Supervisionado

O treinamento é orquestrado pelo script `train_qlora_unsloth.py`, detalhado na Listagem 4.2. O código executa o carregamento em 4 bits, a injeção dos adaptadores LoRA,

Tabela 3 – Hiperparâmetros de Treinamento QLoRA para Qwen 2.5 1.5B.

Hiperparâmetro	**Valor**	**Justificativa Técnica**
Modelo Base	`Qwen/Qwen2.5-1.5B-Instruct`	Eleito pelo melhor equilíbrio throughput/latência em CPU.
Posto LoRA (r)	16	Capacidade representacional suficiente para morfologia e JSON.
Escalar LoRA (α)	32	Razão padrão $\alpha/r = 2$ para estabilidade de gradientes.
Módulos Alvo	Todos os lineares	`q, k, v, o_proj, gate, up, down_proj` para retenção integral.
Taxa de Aprendizado (η)	2×10^{-4}	Taxa recomendada pela literatura de QLoRA para modelos compactos.
Decaimento (*LR Scheduler*)	Cosine com Warmup	5% de passos de aquecimento com decaimento suave a zero.
Tamanho do Lote Efetivo	16	2 amostras por GPU $\times$ 8 passos de acumulação de gradientes.
Épocas de Treinamento	3 épocas	Total de aproximadamente 280 passos de otimização.
Comprimento de Contexto	2048 tokens	Acomoda folgadamente o template ChatML e respostas JSON.
Precisão Numérica	bfloat16 / fp16	A depender do suporte da GPU (bfloat16 em Ampere ou superior).

Fonte: Elaborada pelos autores.

a formatação das mensagens no padrão ChatML e o salvamento final dos pesos adaptados.

```
#!/usr/bin/env python3
"""
Script: train_qlora_unsloth.py
Objetivo: Executar o Fine-Tuning supervisionado via QLoRA 4-bit no Qwen
    2.5 1.5B
utilizando o framework Unsloth e a biblioteca TRL SFTTrainer.
"""

import torch
from unsloth import FastLanguageModel
from datasets import load_dataset
from trl import SFTTrainer
from transformers import TrainingArguments
```

```python

# 1. Configuracao dos hiperparametros
MAX_SEQ_LENGTH = 2048
MODEL_NAME = "unsloth/Qwen2.5-1.5B-Instruct-bnb-4bit"
DATASET_PATH = "dataset_budget_qwen_1500.jsonl"
OUTPUT_DIR = "qwen2.5_budget_1.5b_lora"

print(f" Carregando modelo base: {MODEL_NAME}")
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name=MODEL_NAME,
    max_seq_length=MAX_SEQ_LENGTH,
    dtype=None,              # Detecta automaticamente Float16 ou Bfloat16
    load_in_4bit=True,
)

# 2. Injetar adaptadores LoRA em todas as camadas lineares
print(" Injetando matrizes de baixo posto (LoRA)...")
model = FastLanguageModel.get_peft_model(
    model,
    r=16,
    target_modules=[
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj",
    ],
    lora_alpha=32,
    lora_dropout=0,          # 0 e otimizado nativamente pelo Unsloth
    bias="none",
    use_gradient_checkpointing="unsloth",
    random_state=42,
)

# 3. Preparar e formatar o dataset no ChatML do Qwen
def formatting_prompts_func(examples):
    convos = examples["messages"]
    texts = [tokenizer.apply_chat_template(convo, tokenize=False, add_generation_prompt=False) for convo in convos]
    return {"text": texts}

print(f" Carregando dataset: {DATASET_PATH}")
raw_dataset = load_dataset("json", data_files=DATASET_PATH, split="train")
dataset = raw_dataset.map(formatting_prompts_func, batched=True)

# 4. Parametros do Treinador (SFTTrainer)
training_args = TrainingArguments(
    per_device_train_batch_size=2,
```

```
    gradient_accumulation_steps=8,
    warmup_ratio=0.05,
    num_train_epochs=3,
    learning_rate=2e-4,
    fp16=not torch.cuda.is_bf16_supported(),
    bf16=torch.cuda.is_bf16_supported(),
    logging_steps=10,
    optim="paged_adamw_8bit",
    weight_decay=0.01,
    lr_scheduler_type="cosine",
    seed=42,
    output_dir=OUTPUT_DIR,
    report_to="none"
)

trainer = SFTTrainer(
    model=model,
    tokenizer=tokenizer,
    train_dataset=dataset,
    dataset_text_field="text",
    max_seq_length=MAX_SEQ_LENGTH,
    dataset_num_proc=2,
    packing=False,
    args=training_args,
)

# 5. Execucao do ciclo de Fine-Tuning
print(" Iniciando ciclo de otimizacao...")
trainer_stats = trainer.train()
print(f" Treinamento concluido com sucesso em {trainer_stats.metrics['
    train_runtime']:.2f} segundos!")

# 6. Salvar os adaptadores LoRA treinados
print(f" Salvando adaptadores em: {OUTPUT_DIR}")
model.save_pretrained(OUTPUT_DIR)
tokenizer.save_pretrained(OUTPUT_DIR)
```

Listing 4.2 – Script completo de treinamento com Unsloth (`train_qlora_unsloth.py`).

5 Fusão de Pesos, Conversão GGUF e Empacotamento para Ollama

5.1 Fusão de Pesos LoRA e Exportação Direta

Após o término da otimização supervisionada, as matrizes de posto reduzido B e A devem ser mescladas (*merged*) de volta à matriz original de pesos conforme a Equação (2.1):

$$W_{\text{final}} = W_0 + \frac{\alpha}{r}(B \cdot A) \tag{5.1}$$

Essa operação matemática elimina por completo qualquer sobrecarga computacional durante a inferência, pois o modelo resultante preserva exatamente a mesma arquitetura densa do modelo original, eliminando ramificações de grafos em tempo de execução.

Com o Unsloth, a fusão dos pesos e a exportação direta para o formato binário GGUF quantizado em 4 bits (`q4_k_m`) são realizadas através de um único comando em Python, conforme ilustrado na Listagem 5.1:

```
#!/usr/bin/env python3
"""
Script: export_to_gguf.py
Objetivo: Realizar a fusao dos pesos LoRA e gerar diretamente o arquivo
binario GGUF Q4_K_M para execucao no Ollama em processador Xeon.
"""

from unsloth import FastLanguageModel

MODEL_DIR = "qwen2.5_budget_1.5b_lora"
MAX_SEQ_LENGTH = 2048

print(" Carregando adaptadores LoRA para fusao...")
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name=MODEL_DIR,
    max_seq_length=MAX_SEQ_LENGTH,
    dtype=None,
    load_in_4bit=True,
)

# Exportacao direta para GGUF com quantizacao em bloco Q4_K_M
print(" Gerando binario GGUF quantizado em Q4_K_M...")
model.save_pretrained_gguf(
    "qwen2.5_budget_1.5b_gguf",
    tokenizer,
```

```
    quantization_method="q4_k_m"
)
print(" Exportacao GGUF concluida com sucesso!")
```

Listing 5.1 – Script de fusão e conversão direta para GGUF (`export_to_gguf.py`).

5.2 Criação do Modelfile do Ollama

O binário gerado (denominado `unsloth.Q4_K_M.gguf`) é empacotado no Ollama do servidor Xeon por meio de uma especificação declarativa `Modelfile`.

A Listagem 5.2 expõe o arquivo de configuração final, contendo o template ChatML rígido, a configuração de threads para a CPU Xeon e o *system prompt* mínimo de 22 tokens:

```
# ============================================================================

# Modelfile Oficial: Qwen 2.5 1.5B Budget Assistant (Fine-Tuned)
# Destino: Ollama em CPU Intel Xeon E5-2640 v2 / 8 GB RAM
# ============================================================================


FROM ./qwen2.5_budget_1.5b_gguf/unsloth.Q4_K_M.gguf

TEMPLATE """{{ if .System }}<|im_start|>system
{{ .System }}<|im_end|>
{{ end }}{{ if .Prompt }}<|im_start|>user
{{ .Prompt }}<|im_end|>
{{ end }}<|im_start|>assistant
{{ .Response }}<|im_end|>
"""

# Parametros de Inferencia Rigorosa para CPU
PARAMETER stop "<|im_start|>"
PARAMETER stop "<|im_end|>"
PARAMETER temperature 0.0
PARAMETER top_p 0.9
PARAMETER top_k 40
PARAMETER num_ctx 2048
PARAMETER num_thread 8

# System Prompt Ultraleve (apenas 22 tokens para TTFT < 150 ms)
SYSTEM """Você é o extrator e assistente orçamentário oficial do Estado
    do Paraná."""
```

Listing 5.2 – Arquivo declarativo do Ollama (`Modelfile`).

5.3 Automação da Implantação no Home Lab

Para registrar o modelo no serviço do Ollama em execução no contêiner do Home Lab, elabora-se o script de automação em Bash `deploy_model.sh`, conforme a Listagem 5.3:

```
#!/usr/bin/env bash
set -euo pipefail

CONTAINER_NAME="homelab-slm-ollama"
MODEL_TAG="qwen2.5-budget:1.5b"
MODELFILE_PATH="Modelfile"

echo "
    ========================================================================
    "
echo " Registrando Modelo Customizado no Ollama"
echo " Tag de Destino: ${MODEL_TAG}"
echo "
    ========================================================================
    "

# Copiar os arquivos necessarios para dentro do volume do Ollama
docker cp "${MODELFILE_PATH}" "${CONTAINER_NAME}:/root/Modelfile"
docker cp ./qwen2.5_budget_1.5b_gguf "${CONTAINER_NAME}:/root/"

# Criar o modelo a partir do Modelfile compilado
docker exec -t "${CONTAINER_NAME}" ollama create "${MODEL_TAG}" -f /root
    /Modelfile

echo " Verificando modelos disponíveis no contêiner:"
docker exec -t "${CONTAINER_NAME}" ollama list

echo " Homologação concluída com êxito!"
```

Listing 5.3 – Script Bash para compilação e registro no Ollama (`deploy_model.sh`).

6 Protocolo Experimental de Avaliação e Critérios de Aceitação

6.1 Critérios de Aceitação para Homologação em Produção

A homologação do modelo ajustado e sua promoção para os fluxos ativos no orquestrador n8n dependem do cumprimento irrestrito dos critérios quantitativos discriminados na Tabela 4:

Tabela 4 – Critérios de Aceitação Quantitativos: Pré-Treinamento vs. Pós-Treinamento.

Métrica de Desempenho	**Base (Zero-Shot)**	**Meta (Pós-FT)**	**Impacto Prático no n8n**
Acurácia de Radicais	33,3%	$\geq$ **95,0**%	Eliminação de buscas SQL vazias no PostgreSQL.
Latência de TTFT	$\sim$ 1.200 ms	$<$ **200 ms**	Resposta instantânea ao usuário na CPU.
Vazão ($T_{\mathbf{gen}}$)	13,33 tok/s	$\geq$ **13,0 tok/s**	Fluidez de resposta sem degradação.
Taxa de Alucinação ($H_{\mathbf{rate}}$)	16,7%	**0,0**%	Conformidade estrita na gestão fiscal.
Conformidade JSON	100,0%	**100,0**%	Estabilidade dos nós estruturados do n8n.
Consumo de RAM	1,6 GB RAM	$\leq$ **1,8 GB**	Operação estável no teto de 8 GB do Xeon.

Fonte: Elaborada pelos autores.

6.2 Bateria de Casos de Teste Automatizados

A suíte de testes de validação pós-treinamento reutiliza o executor de benchmarks `run_benchmark.py`, acrescida de novos casos de borda específicos (Tabela 5):

Tabela 5 – Casos de Teste Padronizados para Homologação do Fine-Tuning.

ID	Entrada do Usuário	Comportamento Esperado	SQL?
TC01	*"Olá, bom dia!"*	Resposta institucional de saudação sem parâmetros.	Não
TC02	*"8000"*	Extração: `termo1: 8000, termo2: 8000, termo3: 8000`.	Sim
TC03	*"encargos especiais UEL"*	Normalização: `termo1: encargo, radical1: encar, termo3: uel`.	Sim
TC04	*"saúde pública"*	Remoção diacrítica: `termo1: saude, termo2: publica`.	Sim
TC05	*"reforma de telhado"*	Extração de raízes: `termo1: reforma, radical1: refor`.	Sim
TC06	*"Qual o cardápio do RU hoje?"*	Recusa polida e foco exclusivo em orçamento ($H_{\text{rate}} = 0$).	Não
TC07	*"Como preencher uma DAD para despesa de capital?"*	Resposta normativa orientando a dotação segundo o MTO-PR.	Não

Fonte: Elaborada pelos autores.

7 Checklist Operacional para Execução no Home Lab

Este capítulo consolida o roteiro cronológico passo a passo a ser executado pelo pesquisador assim que restabelecido o acesso presencial ou remoto ao ambiente físico do laboratório doméstico (*Home Lab*):

1. **Verificação da GPU e Ambiente CUDA**:

```
nvidia-smi --query-gpu=name,memory.total,driver_version --format=csv
python3 -c "import torch; print('CUDA Ativa:', torch.cuda.is_available(), '| GPU:', torch.cuda.get_device_name(0))"

```

2. **Criação do Ambiente Virtual e Instalação do Unsloth**:

```
python3 -m venv .venv_finetune
source .venv_finetune/bin/activate
pip install --upgrade pip
pip install "unsloth[colab-new] @ git+https://github.com/unslothai/unsloth.git"
pip install --no-deps xformers "trl<0.9.0" peft accelerate bitsandbytes

```

3. **Geração do Corpus Fiscal de Treinamento (1.500 Pares)**:

```
python3 curate_dataset.py
# Validar se o arquivo dataset_budget_qwen_1500.jsonl possui 1.500 linhas
wc -l dataset_budget_qwen_1500.jsonl

```

4. **Execução do Treinamento QLoRA**:

```
python3 train_qlora_unsloth.py

```

5. **Fusão e Conversão para Binário GGUF**:

```
python3 export_to_gguf.py

```

6. **Registro e Deploy no Contêiner Ollama do Servidor Xeon**:

```
bash deploy_model.sh

```

7. **Execução dos Casos de Teste Automatizados e Homologação**:

```
cd ../testbench/benchmark
python3 run_benchmark.py --models qwen2.5-budget:1.5b --output
   report_post_finetune.json

```

8 Considerações Finais e Desdobramentos

A formulação matemática e o roteiro de engenharia delineados neste documento oferecem uma resposta rigorosa e definitiva aos limites demonstrados pela Engenharia de Prompt no contexto de processamento de linguagem natural voltado à gestão fiscal paranaense.

A estratégia de especialização supervisionada de baixo posto quantizado (QLoRA) sobre o modelo **Qwen 2.5 1.5B Instruct** permite conciliar a estrita soberania de dados públicos e a ausência de infraestrutura computacional de ponta com métricas elevadas de acurácia normativa e extração estruturada.

Ao eliminar a sobrecarga de pré-processamento de contexto no processador Intel Xeon, o sistema atinge tempos de resposta sub-segundo, eliminando alucinações e viabilizando a integração confiável e contínua do assistente orçamentário aos servidores públicos da Universidade Estadual de Londrina e do Estado do Paraná.

Referências

Brasil. *Lei nº 4.320, de 17 de março de 1964. Estatui Normas Gerais de Direito Financeiro para elaboração e controle dos orçamentos e balanços da União, dos Estados, dos Municípios e do Distrito Federal.* 1964. Brasília: Presidência da República. Citado na página 7.

DETTMERS, T. et al. QLoRA: Efficient finetuning of quantized LLMs. In: *Advances in Neural Information Processing Systems (NeurIPS).* [S.l.: s.n.], 2023. v. 36, p. 10088–10115. Citado na página 11.

DUBEY, A. et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. Citado na página 8.

HAN, D.; HAN, M.; Unsloth Team. *Unsloth: Fast and Memory-Efficient Finetuning of Large and Small Language Models.* 2023. <https://github.com/unslothai/unsloth>. Citado na página 19.

HU, E. J. et al. LoRA: Low-rank adaptation of large language models. In: *International Conference on Learning Representations (ICLR).* [S.l.: s.n.], 2022. Citado na página 10.

KIM, S. et al. Full stack optimization of transformer inference on CPUs. *IEEE Micro*, v. 43, n. 5, p. 71–80, 2023. Citado na página 8.

Paraná. Secretaria de Estado da Fazenda. *Manual Técnico do Orçamento (MTO) do Estado do Paraná.* 2026. Curitiba: SEFA/PR. Disponível em: <https://www.fazenda.pr.gov.br/Pagina/Manual-Tecnico-do-Orcamento-MTO>. Citado na página 7.

Paraná. Secretaria de Estado da Fazenda. *Resolução SEFA nº 01/2026: Estabelece normas e procedimentos complementares para a execução orçamentária e financeira.* 2026. Diário Oficial do Estado do Paraná. Citado na página 7.

PostgreSQL Global Development Group. *PostgreSQL 16 Documentation: pg_trgm and unaccent Extensions for Trigram Matching and Diacritic Normalization.* 2024. <https://www.postgresql.org/docs/current/pgtrgm.html>. Citado na página 7.

YANG, A. et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. Citado na página 8.